

Design and Implementation of a 2D Side-Scrolling Shooter Game Using Python and Pygame

[Ankur Bandopadhyay Roll No-23053105]

Department of Computer Science | [Kalinga Institute of Industrial Technology(KIIT)]

[Bhubaneswar, Odisha, India] | [23053105@kiit.ac.in]

Article Info

Keywords: Pygame, Python, Game Development, OOP, Tile Map, AI, Scrolling Shooter

Project Reference: YouTube Tutorial Series — Pygame Shooter (12 Parts)

Tools: Python 3.x, Pygame 2.x, CSV, OS module

Platform: Windows / Linux / macOS

Abstract

This paper documents the design and implementation of a 2D side-scrolling shooter game developed using Python and the Pygame library. The project incorporates core game development concepts including sprite-based animation, tile map construction, physics simulation, enemy artificial intelligence, and multi-level progression. Developed by following and extending a structured twelve-part YouTube tutorial series, the game features a Soldier class used for both player and enemy characters, a World class for tile-based level loading from CSV files, and physics-driven projectile systems for bullets and grenades. The paper presents the architecture, class structure, game loop logic, and implementation details of each component. Results demonstrate a fully functional, multi-level interactive game. The project serves as a comprehensive introduction to object-oriented game development in Python and illustrates how fundamental programming concepts translate into interactive software.

1. INTRODUCTION

The development of interactive video games represents one of the most comprehensive exercises in applied computer science. It requires the simultaneous application of object-oriented programming, physics simulation, graphical rendering, user input handling, and algorithmic problem-solving. Python, with its clean syntax and extensive library support, has emerged as an accessible yet powerful platform for game development, particularly through the Pygame framework.

This paper presents the design and implementation of a 2D side-scrolling shooter game developed entirely in Python using the Pygame library. The game was built by studying and extending a structured twelve-part YouTube tutorial series, authored by a dedicated game development educator. The project involved implementing, adapting, and debugging all core

components — from a scrollable tile-based world to an autonomous enemy AI — resulting in a fully operational multi-level shooter game.

The primary objectives of this project were to gain hands-on experience with event-driven programming, sprite animation, collision detection, and object-oriented design patterns as applied to real-time interactive applications. Beyond merely following a tutorial, the project required deep engagement with the underlying mechanics, debugging logical errors, and extending the code with custom modifications.

The paper is structured as follows: Section 2 provides background on Pygame and tile-based game design. Section 3 describes the overall system architecture. Section 4 details the implementation of each component. Section 5 discusses the results.

2. BACKGROUND AND RELATED CONCEPTS

2.1 Python and Pygame

Python is a high-level, interpreted programming language widely used in education, data science, and software development. Its readability and extensive standard library make it an ideal language for teaching and prototyping. Pygame is an open-source, cross-platform set of Python modules designed for writing video games. It provides functionality for rendering graphics, handling keyboard and mouse input, playing audio, and managing time — all of which are essential for game development. Pygame is built on top of SDL (Simple DirectMedia Layer) and provides an accessible abstraction layer over low-level graphics and input operations.

2.2 Tile-Based Game Design

Tile-based games represent a foundational design paradigm in 2D game development. In this model, the game world is divided into a grid of uniform square tiles, each representing a specific type of terrain, obstacle, or interactive element. This approach simplifies level design, collision detection, and rendering. The game developed in this project uses a 16-row by 150-column tile grid, with 21 distinct tile types, stored as integer values in CSV files. This allows for efficient level design, easy modification, and programmatic loading of levels.

2.3 Object-Oriented Design in Games

Object-oriented programming (OOP) is naturally suited to game development. Each game entity — player, enemy, bullet, grenade — can be modelled as an object with its own properties and behaviours. Pygame's Sprite and Group classes provide a built-in framework for this pattern. The project uses a class hierarchy where the Soldier class serves as both the player character and enemy units, demonstrating effective code reuse through parameterised constructors. Supporting classes such as Bullet, Grenade, Explosion, Itembox, HealthBar, World, and several environmental tile classes form a complete entity model.

2.4 Reference Tutorial Series

The project is based upon a twelve-part YouTube tutorial series covering the development of a complete Pygame shooter game from scratch. The series progressively introduces concepts including display setup, character sprites, animation systems, physics, collision detection, tile maps, enemy AI, weapons, HUD elements, pickups, and multi-level progression. This paper draws on that series as the primary pedagogical reference, with the author's own code constituting the final implemented artefact.

3. SYSTEM DESIGN AND ARCHITECTURE

3.1 Game Configuration Constants

The game is configured through a set of global constants defined at program initialization. These constants establish the dimensions and physical properties of the game environment. The tile size is

derived programmatically from the screen height and row count, ensuring that the tile grid always fills the screen exactly regardless of resolution adjustments. This design choice promotes scalability.

```
SCREEN_WIDTH = 500
SCREEN_HEIGHT = int(SCREEN_WIDTH * 0.8) # 400 px - 80% of width
GRAVITY = 0.75 # pixels/frame²
SCROLL_THRESH = 200 # px before screen scrolls
ROWS = 16 # tile rows in the world grid
TILE_SIZE = SCREEN_HEIGHT // ROWS # 25 px per tile
COLS = 150 # total tile columns per level
TILE_TYPES = 21 # distinct tile image types
MAX_LEVELS = 3 # number of playable levels
```

3.2 Class Architecture

The game is structured around eight primary classes, each inheriting from `pygame.sprite.Sprite` where applicable. The table below summarises the class hierarchy and responsibilities: All sprite classes

register themselves into Pygame Sprite Groups, enabling batch updates and batch rendering through a single `draw()` call. This architectural choice significantly simplifies the main game loop.

Class	Role	Key Methods
Soldier	Player & Enemy characters	<code>move()</code> , <code>shoot()</code> , <code>ai()</code> , <code>update_animation()</code>
World	Level tile map loader	<code>process_data()</code> , <code>draw()</code>
Bullet	Bullet projectile	<code>update()</code> — movement & collision
Grenade	Thrown explosive	<code>update()</code> — physics, timer, blast
Explosion	Visual blast effect	<code>update()</code> — frame animation
Itembox	Pickup items on map	<code>update()</code> — collision & pickup
HealthBar	HUD health display	<code>draw()</code> — ratio-based rendering
Decoration/Water/Exit	Environment tiles	<code>update()</code> — scroll sync

3.3 Game Loop Structure

The main game loop follows the standard update-render pattern of real-time interactive applications. It operates at a fixed 60 frames per second and is structured into five phases: (1) clock tick and frame-rate enforcement, (2) background and world rendering,

(3) entity updates — player, enemies, projectiles, and effects, (4) HUD rendering — health bar, ammo, and grenade display, and (5) event processing — keyboard input, window close, and state transitions. This separation of concerns.

4.METHODOLOGY

The development of the 2D side-scrolling shooter game followed a structured, iterative methodology combining tutorial-based learning with independent implementation and enhancement. The overall approach can be divided into five key phases: requirement analysis, system design, implementation, testing, and refinement.

4.1 Requirement Analysis

The initial phase involved identifying the functional and technical requirements of the game. Core features such as player movement, enemy interaction, projectile systems, tile-based world design, and multi-level progression were defined. The game was designed to operate in a real-time.

4.2 System Design

A modular, object-oriented design approach was adopted. The system was decomposed into multiple classes including `Soldier`, `World`, `Bullet`, `Grenade`, `Explosion`, and `Itembox`. Each class was assigned a specific responsibility, ensuring separation of concerns and ease of maintainability. The tile-based map structure using CSV files was chosen for scalability and flexibility in level design.

4.3 Implementation Strategy

The implementation was carried out incrementally using Python and the Pygame framework. Development began with basic window creation and progressed through sprite rendering, animation handling, and user input

processing. Physics-based movement was implemented using velocity and gravity variables, while collision detection was handled using axis-aligned bounding boxes.

Enemy behaviour was implemented using a simple rule-based artificial intelligence system, incorporating patrol, idle, and attack states. The AI logic was integrated with a vision-based detection mechanism to trigger shooting actions when the player enters a defined range.

4.4 Testing and Debugging

Testing was performed continuously during development using a trial-and-error approach. Key issues such as collision glitches, double-jumping, incorrect sprite alignment, and inconsistent scrolling were identified and

resolved. Debugging techniques included visual inspection, print-based logging, and stepwise execution of movement and collision logic.

4.5 Iterative Refinement

After achieving a functional prototype, the system was refined to improve performance and gameplay experience. Enhancements included optimizing animation timing, implementing cooldown systems for shooting, adding item pickups, and improving enemy behaviour. The level system was expanded to support multiple stages through reusable CSV-based configurations.

This methodology ensured a systematic progression from concept to implementation while allowing flexibility for experimentation and learning.

5. IMPLEMENTATION

5.1 Game Initialization and Window Setup

The game initializes Pygame, creates the display window, loads all image assets, and establishes the frame-rate clock. Images are pre-loaded once into memory at startup to avoid repeated disk access during gameplay. Tile images are stored in a list indexed by tile type number, enabling O(1) lookup

during world rendering. Button images for the main menu and restart screen are also loaded at this stage. This approach ensures smooth and efficient rendering throughout the game. Additionally, pre-loading assets helps minimize lag and improves the overall player experience. The frame-rate clock is set to maintain consistent game speed across different hardware.

```
pygame.init()
screen = pygame.display.set_mode((SCREEN_WIDTH, SCREEN_HEIGHT))
pygame.display.set_caption('Shooter')
clock = pygame.time.Clock()

# Pre-load all 21 tile images into indexed list
img_list = []
for x in range(TILE_TYPES):
    img = pygame.image.load(f'img/Tile/{x}.png')
    img = pygame.transform.scale(img, (TILE_SIZE, TILE_SIZE))
    img_list.append(img)
```

5.2 The Soldier Class — Player and Enemy

The Soldier class is the central entity class of the game. It is used to instantiate both the player character and all enemy units, parameterised by the `char_type` argument ('player' or 'enemy'). Upon initialization, the

class loads four animation sets — Idle, Run, Jump, and Death — by reading image frames from corresponding directory paths. The number of frames per animation is determined dynamically using

os.listdir(), making the system adaptable to sprites with varying frame counts.

Key instance variables include: alive (Boolean health state), speed, ammo, grenades, health, direction (± 1

for flip), vel_y (vertical velocity), in_air (Boolean for ground contact), and animation_list (nested list of animation frames indexed by action and frame).

```
class Soldier(pygame.sprite.Sprite):
    def __init__(self, char_type, x, y, scale, speed, ammo, grenades):
        pygame.sprite.Sprite.__init__(self)
        self.alive = True
        self.char_type = char_type
        self.speed = speed
        self.ammo = ammo
        self.grenades = grenades
        self.health = 100
        self.direction = 1 # 1 = right, -1 = left
        self.vel_y = 0
        self.in_air = True
        self.action = 0 # 0=Idle, 1=Run, 2=Jump, 3=Death
        # AI-specific
        self.vision = pygame.Rect(0, 0, 150, 20)
        self.idling = False
        self.move_counter = 0
```

5.3 Movement, Physics, and Jumping

Movement is handled by the move() method, which accepts Boolean flags moving_left and moving_right from keyboard input. Horizontal displacement (dx) is set to $\pm$ speed accordingly, and the sprite's image is flipped using pygame.transform.flip() to face the direction of travel.

Vertical physics is implemented through a simple Euler integration model. The vertical velocity vel_y is incremented by GRAVITY (0.75 px/frame²) each frame, simulating gravitational acceleration. When the

player jumps, vel_y is set to -11, launching the character upward. Contact with the ground tile surface resets vel_y to zero and clears the in_air flag, preventing double-jumping. Screen scrolling is also managed within the move() method. When the player approaches within SCROLL_THRESH (200 px) of either screen edge, the player's position is held fixed while a screen_scroll offset is returned to the caller, which then shifts all world objects and background layers accordingly.

```
def move(self, moving_left, moving_right):
    # Initialize movement variables
    dx, dy = 0, 0

    # Handle horizontal movement
    if moving_left:
        dx = -self.speed
        self.flip = True # Flip sprite to face left

    if moving_right:
        dx = self.speed
        self.flip = False # Face right

    # Handle jumping (prevent double jump)
    if self.jump and not self.in_air:
        self.vel_y = -11 # Negative value moves character upward
        self.jump = False
        self.in_air = True

    # Apply gravity every frame
    self.vel_y += GRAVITY

    # Limit falling speed (terminal velocity)
    if self.vel_y > 10:
        self.vel_y = 10

    # Update vertical movement
    dy += self.vel_y
```

```
# Return movement values (if needed elsewhere)
return dx, dy
```

5.4 Collision Detection

Collision detection is performed against the `obstaclelist` maintained by the `World` class, which contains all solid tiles (indices 0–8). For each tile, a rectangular collision test is performed separately in the x and y axes. This two-pass approach prevents the character from becoming stuck at corners — a common issue with simultaneous axis collision resolution. Head collisions are detected by checking whether `vely` is negative at the moment of vertical impact, resetting it to zero. Ground landing is detected when `vely` is *non-negative*. This system allows for precise and responsive character movement within the

game environment. By handling collisions axis by axis, the game effectively manages complex interactions with the terrain and obstacles. Additionally, this method supports smooth transitions when the character jumps, falls, or lands, contributing to a more natural gameplay experience. The obstacle list is dynamically updated as the world changes, ensuring collision detection remains accurate throughout different game stages. Overall, this collision framework is essential for maintaining game physics and player control fidelity.

5.5 Enemy Artificial Intelligence

Enemy behaviour is governed by the `ai()` method, which implements a simple finite state machine with three states: patrolling, idling, and attacking. Enemies patrol by moving in their current direction for a distance equal to one tile size before reversing. A random idling mechanism introduces variance — there is a 1-in-200 chance per frame that a patrolling enemy will stop and enter an idle state for 50 frames before resuming movement.

Detection is handled by a vision rectangle (150×20 px) centred at the enemy's position and projected 75 pixels ahead in the direction of travel. If this vision rectangle intersects the player's bounding rectangle, the enemy halts and fires a bullet. The vision rectangle moves with the enemy each frame, ensuring accurate directional awareness.

```
def ai(self):
    if not (self.alive and player.alive):
        return

    # Random idle behavior
    if not self.idling and random.randint(1, 200) == 1:
        self.update_action(0)
        self.idling = True
        self.idling_counter = 50

    # Player detection
    if self.vision.colliderect(player.rect):
        self.update_action(0) # idle/stop
        self.shoot()
    else:
        if not self.idling:
            ai_moving_right = self.direction == 1
            ai_moving_left = not ai_moving_right

            self.move(ai_moving_left, ai_moving_right)
            self.update_action(1) # run
            self.move_counter += 1

        # Update vision
```



```
        self.vision.center = (
            self.rect.centerx + 75 * self.direction,
            self.rect.centery
        )

        # Change direction after moving a tile
        if self.move_counter > TILE_SIZE:
            self.direction *= -1
            self.move_counter *= -1
        else:
            self.idling_counter -= 1
            if self.idling_counter <= 0:
                self.idling = False

        # Apply screen scroll
        self.rect.x += screen_scroll
```

5.6 Weapon Systems — Bullets and Grenades

The Bullet class represents a fired projectile. Each bullet travels horizontally at a speed of 10 pixels per frame in the direction it was fired. The update() method advances the bullet's position, incorporating screen_scroll to keep the bullet correctly positioned relative to the scrolling world. Bullets are destroyed upon leaving the visible screen area, colliding with any obstacle tile, or hitting a character. Player bullets reduce enemy health by 30 HP; enemy bullets reduce player health by 5 HP.

The Grenade class models a thrown explosive with realistic physics. Upon throwing, the grenade is given a horizontal speed of 7 px/frame and an initial vertical velocity of -11 (upward). Gravity is applied each frame. The grenade bounces off walls by reversing its

horizontal direction on collision and stops horizontal movement upon hitting a ceiling. A countdown timer (100 frames) detonates the grenade, spawning an Explosion sprite and dealing 50 HP damage to any entity within a 2-tile blast radius.

5.7 Explosion Visual Effects

The Explosion class renders a sequence of five explosion frames at the detonation point. An internal counter increments each frame, advancing through the frame list at a rate controlled by EXPLOSION_SPEED (4 frames/step). Upon displaying the final frame, the sprite is removed from all groups using the kill() method, freeing memory automatically. This frame-based animation pattern mirrors the technique used for the Soldier's animation.

5.8 Tile Map World System

The World class is responsible for parsing and rendering the tile map. Level data is stored as CSV files (e.g., level1_data.csv), where each cell contains an integer representing a tile type. The process_data() method iterates over this 16×150 matrix and, for each non-negative tile value, creates the appropriate game object. Tiles 0–8 are solid obstacles appended to obstacle_list. Tiles 9–10 are water, 11–14 are

decorations, tile 15 spawns the player, tile 16 spawns enemies, tiles 17–19 create item boxes, and tile 20 places the level exit. Level completion is detected by collision between the player and the Exit sprite. Upon completion, the level counter increments, all sprite groups are cleared via reset_level(), and the next CSV is loaded. If the maximum level is exceeded, the game ends

5.9 Item Boxes and Health Bar (HUD)

Three types of pickup boxes are implemented through the Itembox class: Health (restores 25 HP, capped at maxhealth), Ammo (adds 15 bullets), and Grenade (adds 3 grenades). Collision with the player is detected using pygame.sprite.collidirect(), after which

the appropriate resource is added and the sprite is removed. The HealthBar class renders a dynamic health bar by drawing three overlapping rectangles — a black border, a full red background representing maximum health, and a green foreground whose width

is proportional to current health divided by maximum health. This visual representation provides immediate feedback to the player about their current health status during gameplay. The health bar updates in real-time, reflecting any changes due to damage or healing. The Itembox pickups are strategically placed throughout

the game world to encourage exploration and resource management. Additionally, the removal of the collected item from the game prevents repeated pickups and maintains game balance. This system enhances player engagement by combining visual clarity with interactive gameplay mechanics.

5.10 Parallax Scrolling Background

A parallax scrolling effect simulates depth in the background by moving each background layer at a different speed relative to the camera scroll value. Four layers are used — sky clouds, mountains, near pine trees, and far pine trees — each scrolled at multipliers of 0.5, 0.6, 0.7, and 0.8 respectively.

Layers closer to the viewer scroll faster, creating the illusion of a three-dimensional landscape. Each layer is tiled horizontally across five widths to prevent the background from appearing incomplete as the world scrolls.

```
def draw_bg():
    screen.fill(BG)    # base green fill
    width = sky_img.get_width()
    for x in range(5):
        screen.blit(sky_img, ((x*width) - bg_scroll*0.5, 0))
        screen.blit(mountain_img, ((x*width) - bg_scroll*0.6, ...))
        screen.blit(pine1_img, ((x*width) - bg_scroll*0.7, ...))
        screen.blit(pine2_img, ((x*width) - bg_scroll*0.8, ...))
```

5.11 Main Menu and Button System

A dedicated Button class (imported from button.py) handles interactive UI elements including the Start, Exit, and Restart buttons. Each button stores its image and position, and its draw() method renders the image to the screen while detecting mouse hover and click events. The start_game Boolean flag gates the transition between the main menu state and the active gameplay state within the main loop. When the player dies, the restart button is presented, allowing the

current level to be reloaded without restarting the entire application. This design promotes a seamless user experience by enabling quick retries after failure. The button class also supports visual feedback by changing appearance when hovered over, enhancing interactivity. Furthermore, the modular nature of the Button class allows easy addition or modification of UI elements as the game evolves. This system effectively manages user inputs and game state

5.12 Multi-Level System and Level Progression

Level progression is managed by the global level integer, which begins at 1 and increments upon each level completion. The system supports up to MAX_LEVELS (3) levels, each defined by its own CSV tile map. The reset_level() function empties all sprite groups and returns a fresh empty tile data array.

The level data is then re-read from the appropriate CSV file and a new World object is constructed. This modular approach allows new levels to be added simply by creating additional CSV files, without modifying any game logic code.

6. RESULTS AND DISCUSSION

The completed game is a fully functional, multi-level 2D side-scrolling shooter. The player navigates through tile-based environments, combats autonomous enemy units, collects health, ammo, and grenade

pickups, and progresses through three distinct levels. The game runs stably at the target frame rate of 60 FPS on standard desktop hardware, with smooth scrolling and responsive controls.

6.1 Functional Features Achieved

- Smooth horizontal scrolling with camera threshold and parallax depth layers
- Fully animated player and enemy sprites with Idle, Run, Jump, and Death states
- Physics-accurate jumping and gravity, with solid tile collision detection
- Autonomous enemy AI with patrol, idle, and attack states, and directional vision sensing

Several technical challenges were encountered and resolved during development. Corner collision — where a sprite could become stuck between two tiles — was addressed by separating horizontal and vertical collision resolution into independent passes. Double-jumping was prevented by gating the jump condition on the `in_air` flag. The parallax background required careful calculation of tiling width to avoid visual gaps at different scroll positions.

The current implementation has several limitations. Audio and sound effects are not implemented. The enemy AI, while functional, uses simple heuristic logic and does not incorporate path finding. The game does not include a scoring system, timer, or persistent high-score storage. Collision detection uses rectangular bounding boxes, which may not align precisely with character sprite contours for irregular

- Dual weapon system: rapid-fire bullets and physics-driven grenades with blast radius
- Three types of collectible item boxes dynamically spawned from the tile map
- Dynamic HUD: real-time health bar, bullet count, and grenade count display
- Multi-level progression loaded from CSV tile maps with full state reset between levels
- Main menu with Start/Exit, and in-game Restart upon player death

6.2 Challenges and Solutions

Enemy AI presented a challenge in maintaining directional consistency after reversing. This was resolved by tracking a `move_counter` that accumulates the distance travelled and triggers a direction reversal when a tile-width distance is exceeded. The grenade throw required a check for `grenade_thrown` to prevent continuous spawning while the throw key was held, releasing the flag only on key-up.

6.3 Limitations

shapes. These areas represent opportunities for future enhancement. Additionally, the user interface could be expanded with more interactive elements and better visual feedback. Networked multiplayer functionality is also absent, limiting the game to single-player experiences. Improving these aspects would significantly enrich gameplay depth and player engagement.

7. CONCLUSION

This paper has presented the complete design and implementation of a 2D side-scrolling shooter game built with Python and Pygame. The project demonstrates how object-oriented programming principles, when applied systematically, can produce a complex interactive application from modular, reusable components. The Soldier class illustrates effective code reuse across player and enemy entities. The tile-based World system enables rapid level design through simple CSV data files. Physics and collision systems produce believable movement and interaction, while the enemy AI introduces challenge and unpredictability.

The development process, guided by a structured twelve-part tutorial series and supplemented by independent debugging and customisation, provided comprehensive exposure to real-time event-driven programming, sprite animation, game loop architecture, and state management. The resulting game is fully playable, multi-level, and extensible.

Future work could incorporate pathfinding algorithms such as A* for more intelligent enemy navigation, a tile-map editor for in-game level design, audio effects and background music using Pygame's mixer module, a save-state and scoring system, and network multiplayer using Python's socket library.

R. REFERENCES

- [1] T. Ninja, "Pygame Shooter Game — Part 1: Project Setup & Window," YouTube, 2021. [Online]. Available: <https://youtu.be/DHgj5jhMJKg>
- [2] T. Ninja, "Pygame Shooter Game — Part 2: Player Sprite & Movement," YouTube, 2021. [Online]. Available: <https://youtu.be/7WsE1Vy1Krg>
- [3] T. Ninja, "Pygame Shooter Game — Part 3: Sprite Animation," YouTube, 2021. [Online]. Available: <https://youtu.be/Su5OILSpJDU>
- [4] T. Ninja, "Pygame Shooter Game — Part 4: Scrolling World," YouTube, 2021. [Online]. Available: <https://youtu.be/fEQ8Cle23ko>
- [5] T. Ninja, "Pygame Shooter Game — Part 5: Tile Map & World," YouTube, 2021. [Online]. Available: <https://youtu.be/wqBCWdm9bGY>
- [6] T. Ninja, "Pygame Shooter Game — Part 6: Enemies & AI," YouTube, 2021. [Online]. Available: <https://youtu.be/cS9cagVNSGc>
- [7] T. Ninja, "Pygame Shooter Game — Part 7: Shooting & Bullets," YouTube, 2021. [Online]. Available: <https://youtu.be/IVer6kbsARY>
- [8] T. Ninja, "Pygame Shooter Game — Part 8: Grenades & Explosions," YouTube, 2021. [Online]. Available: <https://youtu.be/1WxQt4-ic3s>
- [9] T. Ninja, "Pygame Shooter Game — Part 9: Health Bar & HUD," YouTube, 2021. [Online]. Available: <https://youtu.be/5VSzZu5904c>
- [10] T. Ninja, "Pygame Shooter Game — Part 10: Item Pickups," YouTube, 2021. [Online]. Available: https://youtu.be/m7GnJo_oZUU
- [11] T. Ninja, "Pygame Shooter Game — Part 11: Level Design & CSV," YouTube, 2021. [Online]. Available: https://youtu.be/JTM8_pcQOUU
- [12] T. Ninja, "Pygame Shooter Game — Part 12: Game Menu & Completion," YouTube, 2021. [Online]. Available: <https://youtu.be/wWLDUb71mFo>
- [13] Pygame Community, "Pygame Documentation v2.x," 2024. [Online]. Available: <https://www.pygame.org/docs/>
- [14] Python Software Foundation, "Python 3 Standard Library — os, csv, random modules," 2024. [Online]. Available: <https://docs.python.org/3/library/>
- [15] R. Nystrom, **Game Programming Patterns**, Genever Benning, 2014. [Online]. Available: <https://gameprogrammingpatterns.com>
- [16] M. Sweigart, **Making Games with Python & Pygame**, CreateSpace, 2012.
- [17] S. Rabin, **Introduction to Game Development**, 2nd ed., Charles River Media, 2010.
- [18] E. Adams, **Fundamentals of Game Design**, 3rd ed., New Riders, 2014.
- [19] J. L. Hennessy and D. A. Patterson, **Computer Architecture: A Quantitative Approach**, 6th ed., Morgan Kaufmann, 2017.
- [20] D. Griffiths and P. Griffiths, **Head First Python**, 2nd ed., O'Reilly Media, 2016.
- [21] Unity Technologies, "2D Game Design Principles," 2023. [Online]. Available: <https://learn.unity.com>
- [22] G. McShaffry and D. Graham, **Game Coding Complete**, 4th ed., Cengage Learning, 2012.

- [23] K. Perlin, "Improving Noise," **ACM Transactions on Graphics**, vol. 21, no. 3, pp. 681–682, 2002.
- [24] A. Rollings and D. Morris, **Game Architecture and Design**, New Riders, 2004.
- [25] SDL Library, "Simple DirectMedia Layer Documentation," 2024. [Online]. Available: <https://www.libsdl.org>
- [26] P. Barry, **Programming Game AI by Example**, Jones & Bartlett Learning, 2004.
- [27] N. Santoro, **Design and Analysis of Algorithms**, Wiley, 2008.
- [28] OpenAI, "Gym Documentation: Reinforcement Learning Environments," 2023. [Online]. Available: <https://gym.openai.com>
- [29] T. Akenine-Möller et al., **Real-Time Rendering**, 4th ed., CRC Press, 2018.
- [30] R. Sedgewick and K. Wayne, **Algorithms**, 4th ed., Addison-Wesley, 2011.